

Pointers in C: Understanding Memory and Address Manipulation

Mohammad Maksood Akhter, PhD
Assistant Professor

Department of Computer Science and Engineering

October 21, 2025



Important

A **pointer** is a variable that is generally used to store the **address of another variable**.

- In pointers, two operators are mainly used:
 - **&** — **Address-of operator**: used to get the address of a variable.
 - ***** — **Dereference (or indirection) operator**: used to access the value stored at a given address.

Pointer Example



```
1  #include <stdio.h>
2
3  void main() {
4      int i = 25;
5      int *ptr;           // ptr is a variable of type integer pointer
6      ptr = &i;           // store the address of i in ptr
7
8      printf("Address of i: %p\n", &i);
9      printf("Address stored in ptr: %p\n", ptr);
10     printf("Value of i: %d\n", i);
11     printf("Value using *ptr: %d\n", *ptr);
12
13     *ptr = 35;           // change value of i through pointer
14
15     printf("After modification:\n");
16     printf("Address of i: %p\n", &i);
17     printf("Value of i: %d\n", i);
18     printf("Value using *ptr: %d\n", *ptr);
19 }
```

Note

- `ptr` is an integer pointer, meaning it can store the address of an integer variable.
- Pointer variables cannot store negative addresses.
- The size of a pointer variable varies from one system to another:
 - 4 bytes on a 32-bit operating system.
 - 8 bytes on a 64-bit operating system.

Important

In this, we will cover the following pointer concepts:

- Basics of pointer
- Pointer to pointer
- Pointer to array
- Pointer to string
- Pointer to function
- Pointer to structure

Important

A pointer can dereference a specific number of bytes depending on its **data type**.

- A `char*` pointer can access **1 byte**.
- An `int*` pointer can access **2 or 4 bytes** (depending on the system).
- A `float*` pointer can access **4 bytes**.

Tip

Each pointer type knows how many bytes to read from memory when dereferenced using the `*` operator.

```
1 #include <stdio.h>
2
3 void main() {
4     int a = 265;
5     char *ptr;
6     int j;
7
8     ptr = (char*)&a; // storing address of integer 'a' into a character pointer
9     j = *ptr;        // dereferencing ptr accesses only 1 byte
10
11     printf("Value of j (without type casting): %d\n", j);
12
13     // Type casting the character pointer to integer pointer
14     j = *(int*)ptr;
15
16     printf("Value of j (after type casting): %d\n", j);
17 }
```

Note

In the above program:

- The address of integer variable `a` is stored in a `char*` pointer `ptr`.
- Since a `char*` accesses only 1 byte, `*ptr` fetches the lowest byte of `a`.
- During compilation, a warning may appear due to pointer type conversion.

To access the complete integer value, perform type casting:

```
j = *(int*)ptr;
```

This temporarily converts `ptr` into an integer pointer, allowing full access to all bytes of `a`. Hence, the correct output after casting will be the original integer value (**265**).

Important

In C programming, there are two types of parameter passing techniques:

- ① Pass by Value (Call by Value)
- ② Pass by Address (Call by Reference)

- In this method, values of variables are passed to the function.
- The function works on copies of these values — changes inside the function do not affect original variables.

Example:

```
1  #include <stdio.h>
2  void swap(int a, int b) {
3      int t;
4      t = a;
5      a = b;
6      b = t;
7  }
8  void main() {
9      int a = 10, b = 20;
10     swap(a, b);
11     printf("%d %d\n", a, b); // Output: 10 20
12 }
```

Explanation: Only copies of `a` and `b` are swapped. Original values remain unchanged.

- Addresses of variables are passed to the function.
- Changes made through pointers affect the original variables.

Example:

```
1  #include <stdio.h>
2  void swap(int *a, int *b) {
3      int t;
4      t = *a;
5      *a = *b;
6      *b = t;
7  }
8  void main() {
9      int a = 10, b = 20;
10     swap(&a, &b);
11     printf("%d %d\n", a, b); // Output: 20 10
12 }
```

Explanation: Passing addresses (&a, &b) allows the function to modify original variables.

Tip

Sometimes, a question may mention “call by reference” even when no explicit address (&) or pointer (*) symbols are used. In such cases, the concept is implicitly assumed — meaning the function is working as if the addresses were passed, and the actual parameters will reflect the changes.

Example:

```
1      #include <stdio.h>
2
3      void n(int x) {
4          x = x * 3;
5          printf("%d ", x);
6      }
7
8      void m(int y) {
9          int a = 18;
10         y = y - a;
11         printf("%d ", y);
12     }
13
14     void main() {
15         int a = 10;
16         n(a);
17         m(a);
18     }
```

Explanation:

- Value of a is passed to both functions (Call by Value)
- n() multiplies x by 3 → prints 30
- m() subtracts 18 from y → prints -8
- Original variable a in main() remains unchanged

Example 1: Basic Pointer to Pointer

```
1 #include <stdio.h>
2
3 void main() {
4     int i = 25;
5     int *ptr;      // pointer to int
6     int **pptr;    // pointer to pointer
7
8     ptr = &i;
9     pptr = &ptr;
10
11     printf("Value of i: %d\n", i);
12     printf("Value using *ptr: %d\n", *ptr);
13     printf("Value using **pptr: %d\n", **pptr)
14     ;
15 }
```

Explanation:

- ptr holds the address of i
- pptr holds the address of ptr
- **pptr accesses the value of i through two levels of indirection

Advanced Example: Multiple Levels of Indirection

```
1  #include <stdio.h>
2  int f(int *x, int *y, int **z) {
3      **z = **z + 3;    // modifies x
4      *y = *y + 2;      // modifies y
5      *x = *x + 3;      // modifies x again
6      return *x + *y;
7  }
8  void main() {
9      int x = 5, y = 10;
10     int *p = &x;
11     int **q = &p;
12
13     int result = f(p, &y, q);
14     printf("Result = %d\n", result);
15 }
```

Explanation:

- p points to x, q points to p
- Function f() receives:
 - x by pointer
 - y by pointer
 - z as pointer to pointer to x
- Nested dereferencing modifies the actual variables

Concept	Description
* ** Pointer to pointer	- Dereference operator (access value at address) - Double dereference (access value through pointer to pointer) - Stores the address of another pointer
Benefit	Allows multi-level memory manipulation (useful in dynamic memory allocation, strings, arrays, and function pointers)

Concept:

- An array name without a subscript (e.g., `a` instead of `a[i]`) represents the base address of the array.
- The base address is constant cannot be changed.
- Hence, an array name is considered a constant pointer.

Example:

```
1 int a[] = {10, 20, 14};
2 int *ptr;
3 ptr = a;      // base address
4 printf("%d\n", *ptr); // 10
5 ptr++;        // move to next element
6 printf("%d\n", *ptr); // 20
7 ptr--;        // move back
8 printf("%d\n", *ptr); // 10
```

- Pointer arithmetic moves the pointer according to the data type size.
- Example: `int a[] = {10,20,30,40}; int *ptr = a;`
 - `ptr + 1` → next element (+4 bytes if `int=4`)
 - `ptr + 2` → two elements forward
- Pointer subtraction: only valid within same array → returns number of elements.

Example:

```
1  int a[] = {10,20,30,40,50};  
2  int *ptr1 = &a[4]; // 50  
3  int *ptr2 = &a[0]; // 10  
4  int diff = ptr1 - ptr2;  
5  printf("%d\n", diff); // 4
```

```
1 #include <stdio.h>
2 int main() {
3     int a[5] = {10,20,30,40,50};
4     int *ptr = a;
5
6     printf("%d ", *a);           // 10
7     printf("%d ", *(a+2));       // 30
8     printf("%d ", a[2]);         // 30
9     printf("%d ", *ptr);         // 10
0     printf("%d\n", *(ptr+4));    // 50
1     return 0;
2 }
```

Output: 10 30 30 10 50

Notes:

- a → constant pointer (cannot change address)
- ptr → pointer variable (can move)
- Pointer addition/subtraction valid only within the same array
- Pointer difference → number of elements, not bytes

Declaration / Expression	Type	Meaning / Description
<code>int a[5];</code>	Array	Array of 5 integers
<code>int *p;</code>	Pointer	Points to an integer
<code>int **pp;</code>	Pointer to pointer	Points to a pointer to int
<code>int *a[5];</code>	Array of pointers	Array of 5 integer pointers
<code>int (*p)[5];</code>	Pointer to array	Pointer to an array of 5 ints
<code>int (*p)(int,int);</code>	Pointer to function	Points to function returning int
<code>const int *p;</code>	Pointer to const	Value cannot be changed
<code>int * const p;</code>	Const pointer	Address cannot be changed
<code>a[2]</code> or <code>2[a]</code>	Array element	Access the 3rd element of array a; <code>2[a]</code> is valid and equivalent to <code>a[2]</code>

- Start from the variable name.
- Move right: if `[]` or `()` → array or function.
- Move left: if `*` → pointer to.
- Repeat until the declaration makes sense.

Example:

- `int (*p)[5];` → Start at `p` → left `*` = pointer → right `[5]` = array of 5 → of `int`.
- Meaning: “p is a pointer to an array of 5 integers.”

Thank You

Questions?